

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Обработка изображений в ИС»

Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:
Студент 4 курса
Группы ИИ-26
Шоева Е.Т.
Проверила:
Андренко К.В.

Цель: научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения

Общее задание

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать **torchvision.datasets**). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№ варианта	Выборка	Размер исходного изображения	Оптимизатор
11	MNIST	28X28	Adadelata

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

```
import matplotlib.pyplot as plt
import numpy as np
import random
import os
from PIL import Image
```

```
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
BATCH_SIZE = 64
TEST_BATCH_SIZE = 1000
EPOCHS = 10
LEARNING_RATE = 1.0
RANDOM_SEED = 42
```

```
torch.manual_seed(RANDOM_SEED)
random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)
```

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
```

```
train_dataset = datasets.MNIST(
```

```

    root="./data", train=True, download=True, transform=transform
)
test_dataset = datasets.MNIST(
    root="./data", train=False, download=True, transform=transform
)

```

```

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=TEST_BATCH_SIZE, shuffle=False)

```

```

class SimpleCNN(nn.Module):
    def __init__(self, num_classes: int = 10):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=16,
                                kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32,
                                kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        self.fc1 = nn.Linear(32 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, num_classes)

        self.dropout = nn.Dropout(0.25)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

```

```

model = SimpleCNN().to(DEVICE)
print(model)

```

```

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adadelta(model.parameters(), lr=LEARNING_RATE)

```

```

def train_one_epoch(epoch: int):
    model.train()
    running_loss = 0.0
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(DEVICE), target.to(DEVICE)

        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()

```

```

optimizer.step()

running_loss += loss.item()

if batch_idx % 200 == 0:
    print(f"Эпоха {epoch} [{batch_idx * len(data)} / {len(train_loader.dataset)}] "
          f"\tLoss: {loss.item():.4f}")

avg_loss = running_loss / len(train_loader)
return avg_loss

def evaluate(loader):
    model.eval()
    total_loss = 0.0
    correct = 0
    with torch.no_grad():
        for data, target in loader:
            data, target = data.to(DEVICE), target.to(DEVICE)
            output = model(data)
            total_loss += criterion(output, target).item() * data.size(0)
            pred = output.argmax(dim=1)
            correct += pred.eq(target).sum().item()

    avg_loss = total_loss / len(loader.dataset)
    accuracy = 100.0 * correct / len(loader.dataset)
    return avg_loss, accuracy

train_losses = []
test_losses = []
test_accuracies = []

for epoch in range(1, EPOCHS + 1):
    train_loss = train_one_epoch(epoch)
    test_loss, test_acc = evaluate(test_loader)

    train_losses.append(train_loss)
    test_losses.append(test_loss)
    test_accuracies.append(test_acc)

    print(f"== Эпоха {epoch}: train_loss={train_loss:.4f}, "
          f"test_loss={test_loss:.4f}, test_acc={test_acc:.2f}% ==\n")

print(f"Итоговая точность на тестовой выборке: {test_accuracies[-1]:.2f}%")

plt.figure(figsize=(8, 5))
plt.plot(range(1, EPOCHS + 1), train_losses, label="Train loss", marker="o")
plt.plot(range(1, EPOCHS + 1), test_losses, label="Test loss", marker="o")
plt.xlabel("Эпоха")
plt.ylabel("Loss (CrossEntropyLoss)")

```

```
plt.title("Изменение ошибки в процессе обучения (MNIST, Adadelata)")
plt.legend()
plt.grid(True)
plt.savefig("loss_curve.png", dpi=150, bbox_inches="tight")
plt.show()
```

```
plt.figure(figsize=(8, 5))
plt.plot(range(1, EPOCHS + 1), test_accuracies, label="Test accuracy", color="green",
marker="o")
plt.xlabel("Эпоха")
plt.ylabel("Точность, %")
plt.title("Точность на тестовой выборке")
plt.grid(True)
plt.savefig("accuracy_curve.png", dpi=150, bbox_inches="tight")
plt.show()
```

```
def visualize_prediction(model, dataset, index: int = None):
```

```
    model.eval()
    if index is None:
```

```
        index = random.randint(0, len(dataset) - 1)
```

```
    image, true_label = dataset[index]
```

```
    input_tensor = image.unsqueeze(0).to(DEVICE)
```

```
    with torch.no_grad():
```

```
        output = model(input_tensor)
```

```
        probabilities = F.softmax(output, dim=1).cpu().numpy().flatten()
```

```
        predicted_label = int(np.argmax(probabilities))
```

```
    img_np = image.squeeze().numpy() * 0.3081 + 0.1307
```

```
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

```
    axes[0].imshow(img_np, cmap="gray")
```

```
    axes[0].set_title(f"Истинная метка: {true_label}\n"
```

```
                    f"Предсказание сети: {predicted_label}")
```

```
    axes[0].axis("off")
```

```
    axes[1].bar(range(10), probabilities)
```

```
    axes[1].set_xticks(range(10))
```

```
    axes[1].set_xlabel("Класс (цифра)")
```

```
    axes[1].set_ylabel("Вероятность")
```

```
    axes[1].set_title("Распределение вероятностей по классам")
```

```
plt.tight_layout()
```

```
plt.savefig("prediction_example.png", dpi=150, bbox_inches="tight")
```

```
plt.show()
```

```
print(f"Индекс изображения: {index}")
```

```
print(f"Истинная метка: {true_label}, Предсказанная метка: {predicted_label}")
```

```
print(f"Вероятности по классам: {np.round(probabilities, 3)}")
```

```

def load_custom_image(path: str, invert: bool = True):
    image = Image.open(path).convert("L")
    image = image.resize((28, 28))
    img_array = np.array(image).astype(np.float32) / 255.0

    if invert:
        img_array = 1.0 - img_array

    img_array = (img_array - 0.1307) / 0.3081
    tensor = torch.tensor(img_array, dtype=torch.float32).unsqueeze(0).unsqueeze(0)
    return tensor, img_array


def predict_custom_image(model, path: str, invert: bool = True):
    model.eval()
    tensor, img_array = load_custom_image(path, invert=invert)
    tensor = tensor.to(DEVICE)

    with torch.no_grad():
        output = model(tensor)
        probabilities = F.softmax(output, dim=1).cpu().numpy().flatten()
        predicted_label = int(np.argmax(probabilities))

    fig, axes = plt.subplots(1, 2, figsize=(10, 4))

    axes[0].imshow(img_array, cmap="gray")
    axes[0].set_title(f"Внешнее изображение\nПредсказание сети: {predicted_label}")
    axes[0].axis("off")

    axes[1].bar(range(10), probabilities)
    axes[1].set_xticks(range(10))
    axes[1].set_xlabel("Класс (цифра)")
    axes[1].set_ylabel("Вероятность")
    axes[1].set_title("Распределение вероятностей по классам")

    plt.tight_layout()
    plt.savefig("custom_prediction.png", dpi=150, bbox_inches="tight")
    plt.show()

    print(f"Файл: {path}")
    print(f"Предсказанная метка: {predicted_label}")
    print(f"Вероятности по классам: {np.round(probabilities, 3)}")

    return predicted_label, probabilities


visualize_prediction(model, test_dataset)

custom_image_path = "my_digit.png"
if os.path.exists(custom_image_path):

```

```

predict_custom_image(model, custom_image_path, invert=True)
else:
    print(f"Файл {custom_image_path} не найден. "
          f"Положи своё изображение цифры рядом со скриптом и укажи его имя, "
          f"чтобы проверить сеть на произвольной картинке.")
== Эпоха 8: train_loss=0.0164, test_loss=0.0295, test_acc=99.17% ==

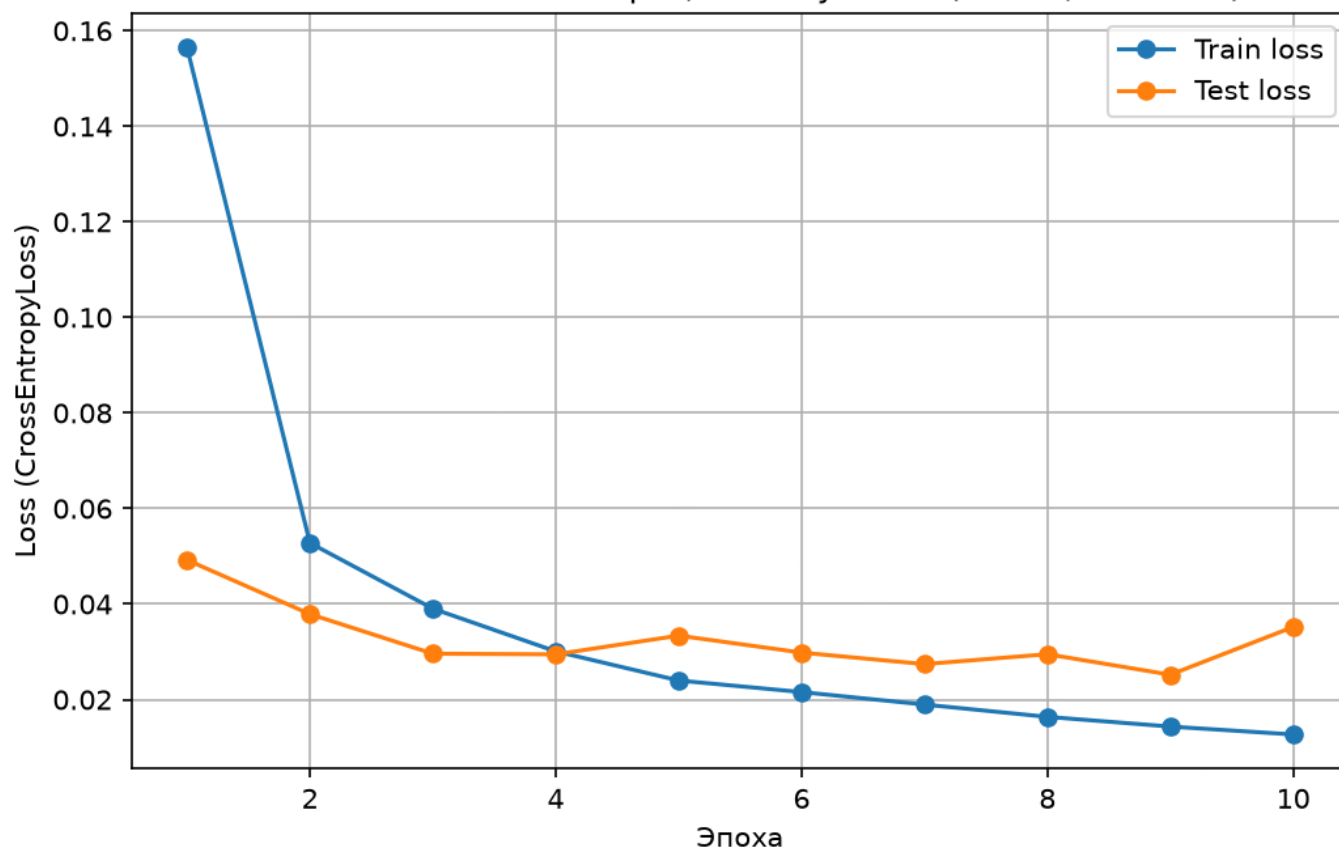
Эпоха 9 [0/60000]   Loss: 0.0382
Эпоха 9 [12800/60000]   Loss: 0.0018
Эпоха 9 [25600/60000]   Loss: 0.0002
Эпоха 9 [38400/60000]   Loss: 0.1266
Эпоха 9 [51200/60000]   Loss: 0.0002
== Эпоха 9: train_loss=0.0144, test_loss=0.0252, test_acc=99.25% ==

Эпоха 10 [0/60000]   Loss: 0.0044
Эпоха 10 [12800/60000]   Loss: 0.0004
Эпоха 10 [25600/60000]   Loss: 0.0214
Эпоха 10 [38400/60000]   Loss: 0.0026
Эпоха 10 [51200/60000]   Loss: 0.0000
== Эпоха 10: train_loss=0.0127, test_loss=0.0352, test_acc=99.12% ==

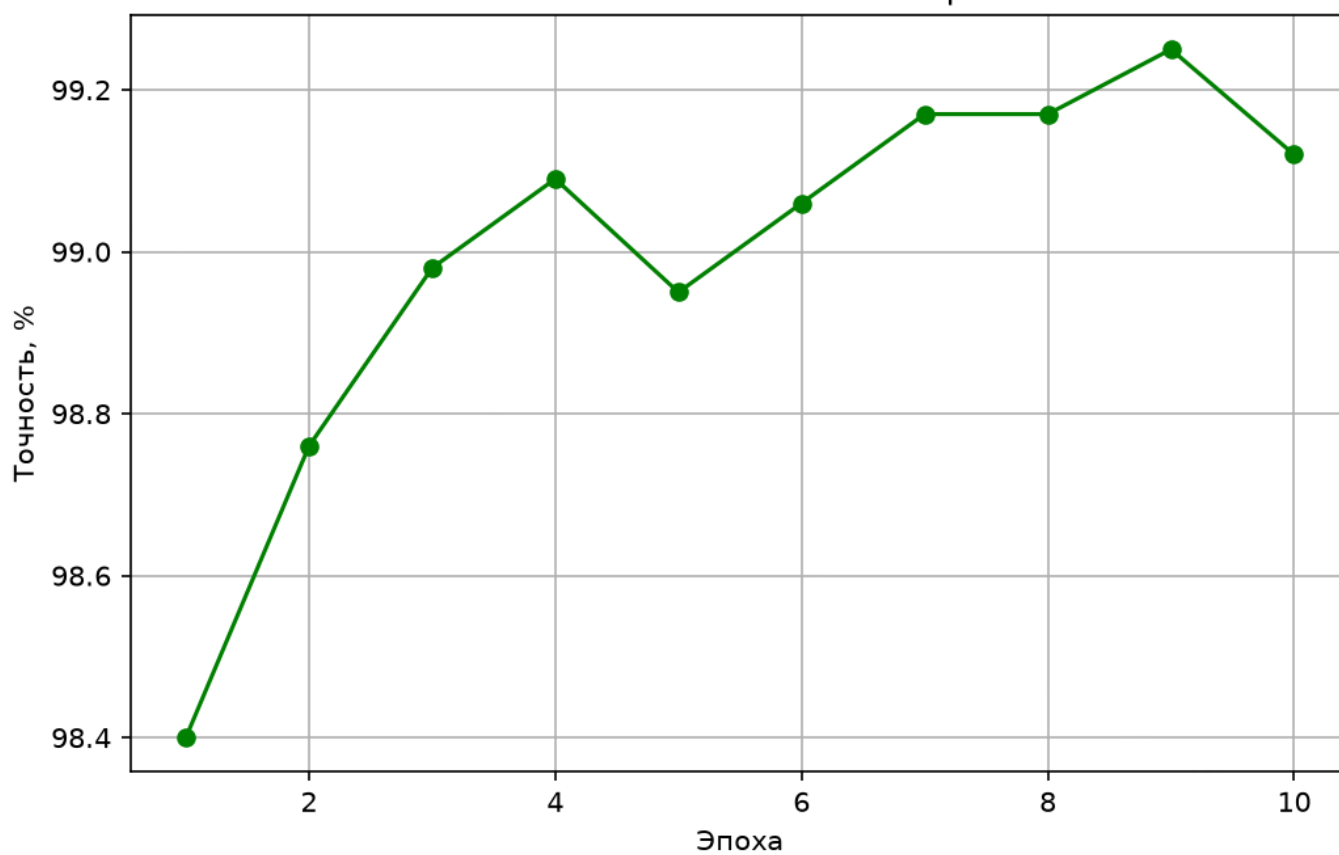
Итоговая точность на тестовой выборке: 99.12%
Индекс изображения: 1824
Истинная метка: 2, Предсказанная метка: 2
Вероятности по классам: [0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
Файл: my_digit.png
Предсказанная метка: 0
Вероятности по классам: [0.995 0. 0. 0. 0. 0. 0. 0. 0. 0.004]

```

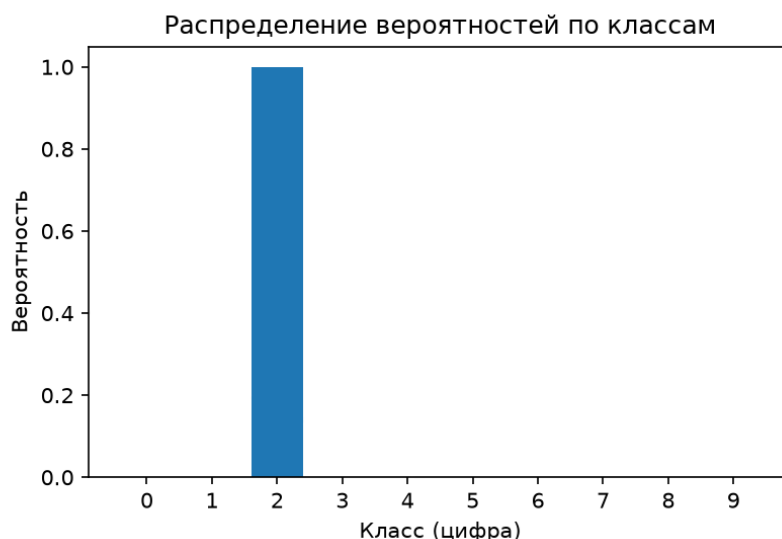
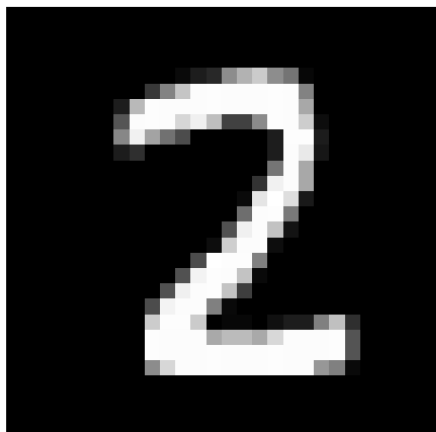
Изменение ошибки в процессе обучения (MNIST, Adadelata)



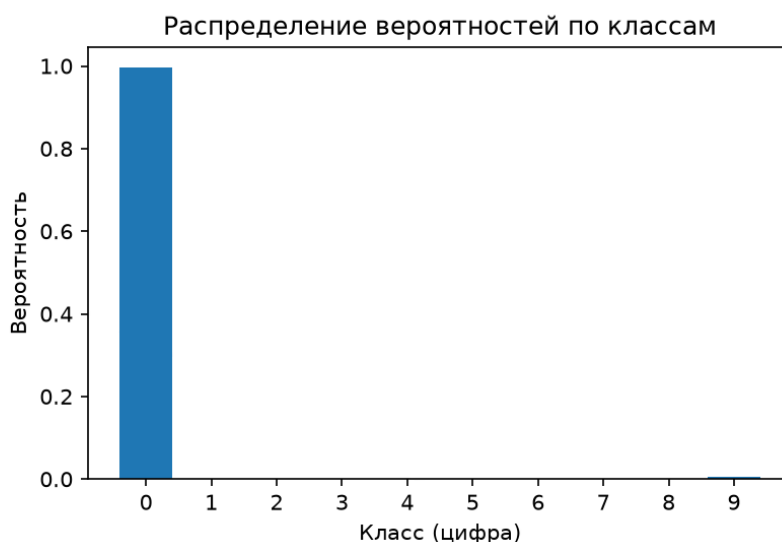
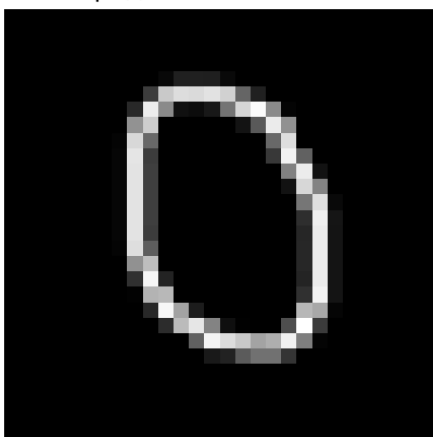
Точность на тестовой выборке



Истинная метка: 2
Предсказание сети: 2



Внешнее изображение
Предсказание сети: 0



Сеть успешно обучилась точность на тестовой выборке выросла с 98.4% до 99.1–99.25%, а ошибка (loss) быстро снизилась и стабилизировалась уже после 3–4 эпохи, при этом небольшой разброс test loss на поздних эпохах говорит о лёгком переобучении, не критичном для итогового качества. На тестовом изображении MNIST (цифра «2») сеть дала верное и уверенное предсказание с вероятностью, близкой к 1.0. При этом на собственном нарисованном изображении (цифра «0») модель тоже справилась верно, что подтверждает способность сети обобщать за пределы обучающей выборки

Вывод: Изучила принципы бинарной классификации и реализовала однослойную нейронную сеть (персептрон) для решения задачи классификации с использованием **пороговой** функции активации, а также исследовать процесс обучения модели с применением среднеквадратичной ошибки (MSE).